

Functions and Data Structures

- ✔ **Video:** Functions
5 min
- ✔ **Video:** Variable scope
4 min
- ✔ **Reading:** Function and variable scope
15 min
- 📖 **Reading:** What are data structures?
10 min
- ▶ **Video:** Lists
5 min
- ▶ **Video:** Tuples
3 min
- ▶ **Video:** Sets
4 min
- ▶ **Video:** Dictionaries
6 min
- ▶ **Video:** kwargs
2 min
- 📖 **Practice Quiz:** Functions, loops and data structures
5 questions
- 📖 **Reading:** Choosing and using data structures
15 min
- 📖 **Reading:** Visual Studio Code on Coursera
10 min
- 📝 **Programming Assignment:** Functions, loops and data structures
3h
- 📖 **Practice Quiz:** Knowledge check: Functions and Data structures
5 questions
- 📖 **Reading:** Additional resources
5 min

Errors, exceptions and file handling

Function and variable scope

Functions and variables

It is essential to understand the levels of scope in Python and how things can be accessed from the four different scope levels. Below are the four scope levels and a brief explanation of where and how they are used.

1. Local scope

Local scope refers to a variable declared inside a function. For example, in the code below, the variable `total` is only available to the code within the `get_total` function. Anything outside of this function will not have access to it.

```
1 def get_total(a, b):
2     #local variable declared inside a function
3     total = a + b;
4     return total
5
6 print(get_total(5, 2))
7 7
8
9 # Accessing variable outside of the function:
10 print(total)
11 NameError: name 'total' is not defined
```

Run
Reset

Error on line 11:
NameError: name 'total' is not defined
SyntaxError: invalid syntax

2. Enclosing scope

Enclosing scope refers to a function inside another function or what is commonly called a **nested function**.

In the code below, I added a nested function called `double_it` to the `get_total` function.

As `double_it` is inside the scope for the `get_total` function it can then access the variable. However, the enclosed variable inside the `double_it` function cannot be accessed from inside the `get_total` function.

```
1 def get_total(a, b):
2     #enclosed variable declared inside a function
3     total = a + b
4
5     def double_it():
6         #local variable
7         double = total * 2
8         print(double)
9
10    double_it()
11    #double variable will not be accessible
12    print(double)
13
14    return total
```

Run
Reset

No Output

3. Global scope

Global scope is when a variable is declared outside of a function. This means it can be accessed from anywhere.

In the code below, I added a global variable called `special`. This can then be accessed from both functions `get_total` and `double_it`:

```
1
2 special = 5
3
4 def get_total(a, b):
5     #enclosed scope variable declared inside a function
6     total = a + b
7     print(special)
8
9     def double_it():
10        #local variable
11        double = total * 2
12        print(special)
13
14    double_it()
15
16    return total
```

Run
Reset

No Output

4. Built-in scope

Built-in scope refers to the reserved keywords that Python uses for its built-in functions, such as `print`, `def`, `for`, `in`, and so forth. Functions with built-in scope can be accessed at any level.

✔ Completed Go to next item

👍 Like 🗨 Dislike 📄 Report an issue

